Home SOC Lab:

Attack Detection and Incident Investigation with Wazuh

A Self-Directed Cybersecurity Project

By Aniket Shahi:

Linkedin: https://www.linkedin.com/in/aniket-shahi-126618375/

GitHub: github.com/shahianiket20

November 2026

Table of Contents:

Table of Contents 2
1. Introduction 3
2. Problem Statement 3
3. Aim and Objectives 4
Objectives 4
4. Methodology 4
5. Tools and Technologies 5
6. System Design / Architecture 5
7. Implementation 7
7.1 Wazuh Server Deployment 7
7.2 Endpoint Deployment 7
7.3 Challenges Encountered During Setup 7
8. Testing 8
8.1 Test 1 — SSH Brute Force 9
8.2 Test 2 — Nmap Port Scan (Detection Gap) 11
8.3 Test 3 — File Integrity Monitoring (FIM) 13
8.4 Test 4 — Privilege Escalation (sudo) 16
9. Detection Engineering: Addressing the Nmap Detection Gap 18
10. Incident Reports 19
Incident Report 1 — SSH Brute Force Attack 19
Incident Report 2 — Unauthorized File Modification (FIM) 19
Incident Report 3 — Privilege Escalation via Sudo 20
Incident Note 4 — Detection Gap: Nmap Port Scan 20
11. Results and Analysis 21
12. Limitations and Future Work 21
13. Conclusion 22
14. References 22

1. Introduction:

Security Operations Centers (SOCs) are responsible for monitoring an organisation's systems, detecting suspicious or malicious activity, and responding to incidents before they cause serious harm. A SOC analyst's day-to-day work revolves around a Security Information and Event Management (SIEM) platform, which collects logs from across an environment, raises alerts when something looks wrong, and provides the evidence needed to investigate and respond.

This project builds a small, self-contained SOC environment on a personal laptop. It uses Wazuh, an open-source SIEM and extended detection and response (XDR) platform, to monitor a Linux endpoint while that endpoint is deliberately attacked from a separate machine. The goal is to gain hands-on experience with the full detection lifecycle: generating an attack, observing how (or whether) it is detected, investigating the resulting alert, and documenting the incident as a SOC analyst would.

This project complements an earlier project, a Cowrie honeypot and attack-monitoring dashboard built as a university final year project, which focused on attracting and recording attacker behaviour. Where that project was about collecting attack data, this project is about detecting and investigating attacks on a system that is meant to be protected the core responsibility of a SOC analyst.

2. Problem Statement:

Without active monitoring, attacks against a system can go unnoticed. A failed login attempt, a port scan, or a modified system file means little in isolation, but a monitoring system that collects and correlates this activity can turn it into a timely, actionable alert.

Many students and entry-level cybersecurity candidates have experience with offensive tools (such as Nmap, Metasploit, and Kali Linux) but limited hands-on experience with the defensive side: setting up a SIEM, tuning detection rules, and investigating the alerts those tools produce. This project addresses that gap directly by building and testing a working detection environment from scratch, on limited hardware (an 8GB RAM machine), reflecting real constraints faced when learning security operations independently.

3. Aim and Objectives:

Aim: To design, build, and test a small-scale Security Operations Centre (SOC) environment capable of detecting and reporting on common attack techniques against a monitored Linux host.

Objectives:

- Deploy a Wazuh SIEM server and connect a monitored Linux endpoint to it.
- Simulate a range of realistic attack techniques against the endpoint from a separate attacker machine.
- Evaluate which attacks Wazuh detects by default, and investigate any detection gaps.
- Design and test a custom detection rule to address an identified gap.
- Produce incident reports for each detected event, in the style used by SOC analysts.
- Document the full process, including challenges encountered and how they were resolved.

4. Methodology:

The project followed an iterative, hands-on methodology rather than a purely theoretical one. Each stage was completed, tested, and documented before moving to the next:

- 1. Planning : selecting Wazuh as the SIEM platform and defining the lab topology.
- 2. Environment setup : building the virtual machines and installing the Wazuh server.
- 3. Endpoint deployment : creating a monitored endpoint and connecting the Wazuh agent.
- 4. Attack simulation : running a series of controlled attacks against the endpoint only.
- 5. Detection analysis : reviewing the Wazuh dashboard for alerts generated by each attack.
- 6. Detection engineering : writing a custom rule to address an identified detection gap.
- 7. Incident reporting : documenting each detected event as a short, structured incident report.
- 8. Write-up and publication : compiling this report and publishing the project to GitHub and LinkedIn.

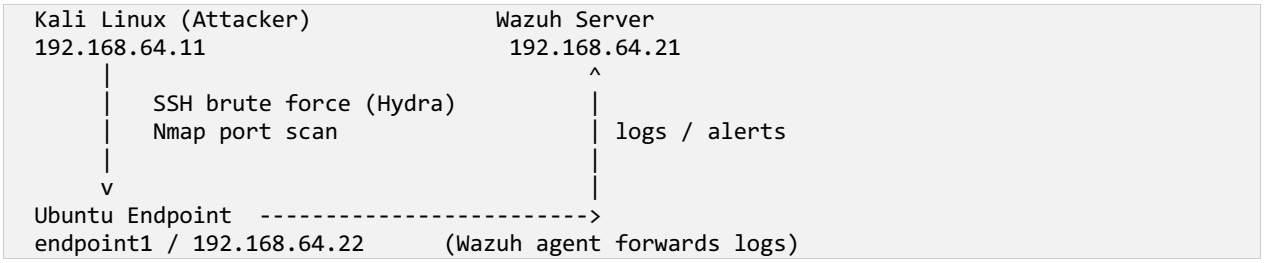
All attacks were carried out exclusively against machines owned and controlled by the author within an isolated virtual network, with no external systems involved at any point.

5. Tools and Technologies:

Tool / Technology	Category	Purpose in this project
UTM (QEMU)	Virtualization	Runs all virtual machines on macOS (Apple Silicon / ARM64)
Ubuntu Server 22.04 (ARM64)	Operating System	Base OS for the Wazuh server and the monitored endpoint
Wazuh 4.14	SIEM / XDR	Collects logs, generates alerts, provides the dashboard
Kali Linux	Attacker machine	Source of simulated attacks (Hydra, Nmap)
Hydra	Attack tool	SSH password brute-forcing
Nmap	Attack / recon tool	Port scanning against the endpoint
UFW	Host firewall	Logs blocked connection attempts for the custom detection rule
OpenSSH	Remote access	Administering all VMs from the host Mac via Terminal
tmux	Terminal multiplexer	Keeps long-running installs alive across SSH disconnects

6. System Design / Architecture:

The lab consists of three virtual machines, running under UTM (QEMU) on an Apple Silicon Mac, connected via a shared virtual network (192.168.64.0/24):



- Wazuh Server: Ubuntu Server 22.04 (ARM64), 4GB RAM, 40GB disk. Runs the Wazuh indexer, manager, Filebeat, and dashboard.
- Endpoint: Ubuntu Server 22.04 (ARM64), 1GB RAM. Runs the Wazuh agent and is the sole target of all attacks.
- Attacker: Kali Linux. Used only to generate attack traffic against the endpoint.



Figure: Active output

7. Implementation:

7.1 Wazuh Server Deployment:

A fresh Ubuntu Server 22.04 (ARM64) virtual machine was created in UTM, with LVM disabled during installation to ensure the full 40GB disk was usable (the default guided LVM layout only allocates roughly half the disk). OpenSSH was installed during setup to allow remote administration from the host Mac's Terminal, which was essential since the UTM console has no copy-paste or mouse support.

7.2 Endpoint Deployment:

The endpoint VM was created by cloning the server VM before Wazuh was installed on it, then giving the clone its own identity: a new hostname (endpoint1), a new machine-id, new SSH host keys, and critically a new network MAC address, since UTM's clone operation otherwise duplicates the original NIC's MAC address, which caused IP conflicts on the shared network.

The Wazuh agent was installed on the endpoint and registered against the server (192.168.64.21), after which the dashboard confirmed one active agent (endpoint1).

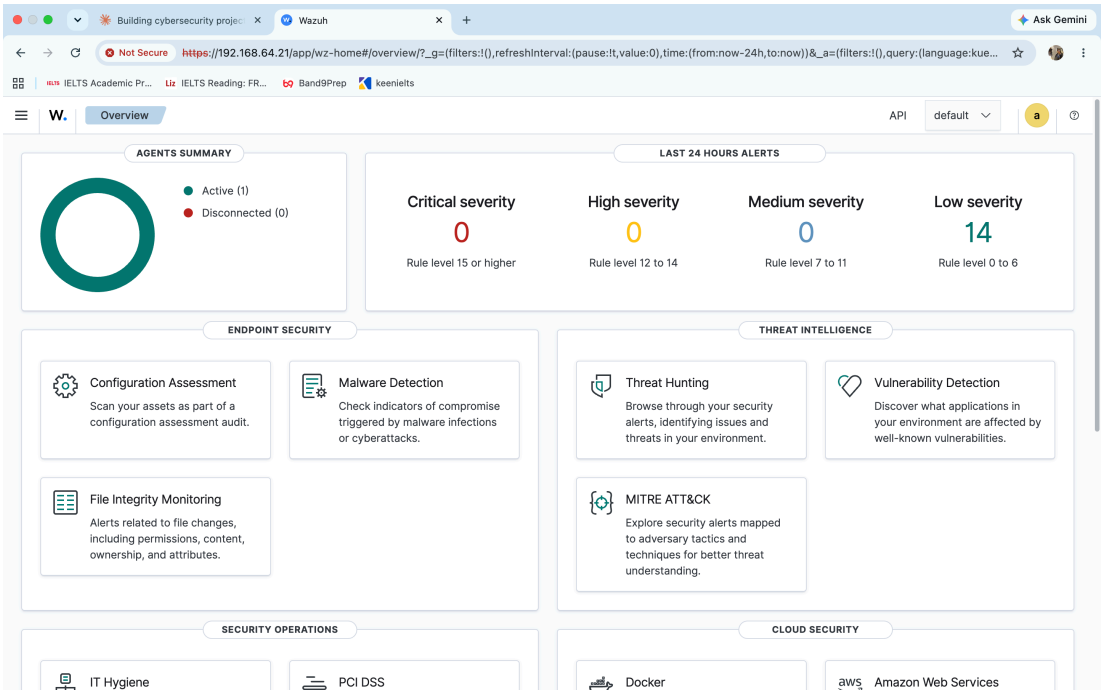


Figure: Wazuh dashboard showing endpoint1 as an active, connected agent

7.3 Challenges Encountered During Setup:

Several real issues were encountered and resolved during the build, which are documented here as they reflect genuine troubleshooting experience:

- Unreliable home internet connection: the Wazuh manager package (560MB) repeatedly failed to download with a connection timeout. Resolved by increasing apt's retry count and timeout, and by running the installation over a more stable network connection.
- Interrupted package installation: a prior failed install left the dpkg package manager in a half-configured state, blocking all further installation. Resolved by running sudo dpkg --configure -a before retrying.
- Duplicate MAC address: cloning the server VM to create the endpoint copied the original network adapter's MAC address, risking IP/network conflicts. Resolved by assigning a new random MAC address to the endpoint VM in UTM before first boot.
- Limited RAM (8GB total): only one or two VMs could run at a time comfortably. Resolved by giving the endpoint a minimal 1GB RAM allocation, running the attacker VM only when actively testing, and keeping the pre-existing honeypot VM powered off during this project.

8. Testing:

Four categories of attack were simulated against the endpoint, covering credential attacks, reconnaissance, file-based tampering, and privilege escalation. Each test is summarised below; full details are provided in the incident reports in Section 10.

ID	Attack / Test	Method / Command	Expected Result	Actual Result
T1	SSH Brute Force	hydra -l ubuntu -P passwords.txt ssh://endpoint1	Alert raised for repeated failed logins	Rule 5763 fired (level 10), MITRE T1110
T2	Nmap Port Scan	nmap -sV / -p 1-200 against endpoint1	Alert raised for scanning behaviour	No alert under default rules; custom rule attempted but did not reliably fire
T3	File Integrity Monitoring	Create, chmod 777, modify, delete a file in a watched folder	Alerts for file added / modified / deleted	All 3 actions detected and

ID	Attack / Test	Method / Command	Expected Result	Actual Result
				logged separately
T4	Privilege Escalation (sudo)	sudo whoami, sudo cat /etc/shadow, etc.	Alert for sudo usage / escalation	Rule 5402 fired; first-time

8.1 Test 1 : SSH Brute Force

A password list of common/incorrect passwords was used with Hydra to attempt repeated SSH logins to the endpoint as user 'ubuntu'. Wazuh detected this immediately, raising progressively higher-severity alerts as failed attempts accumulated, culminating in a dedicated brute-force rule (ID 5763, level 10) mapped to MITRE ATT&CK technique T1110 (Brute Force).

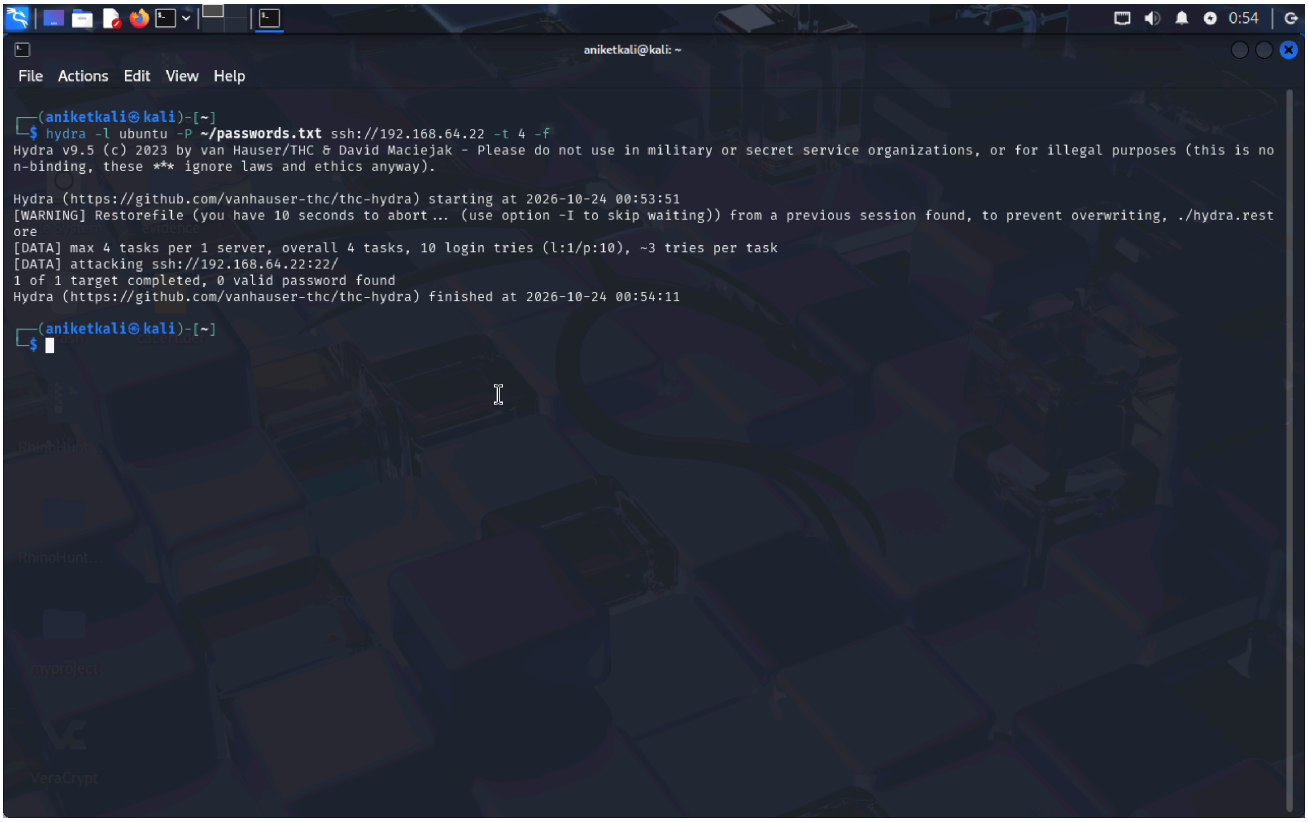


Figure: Hydra command and output against the endpoint

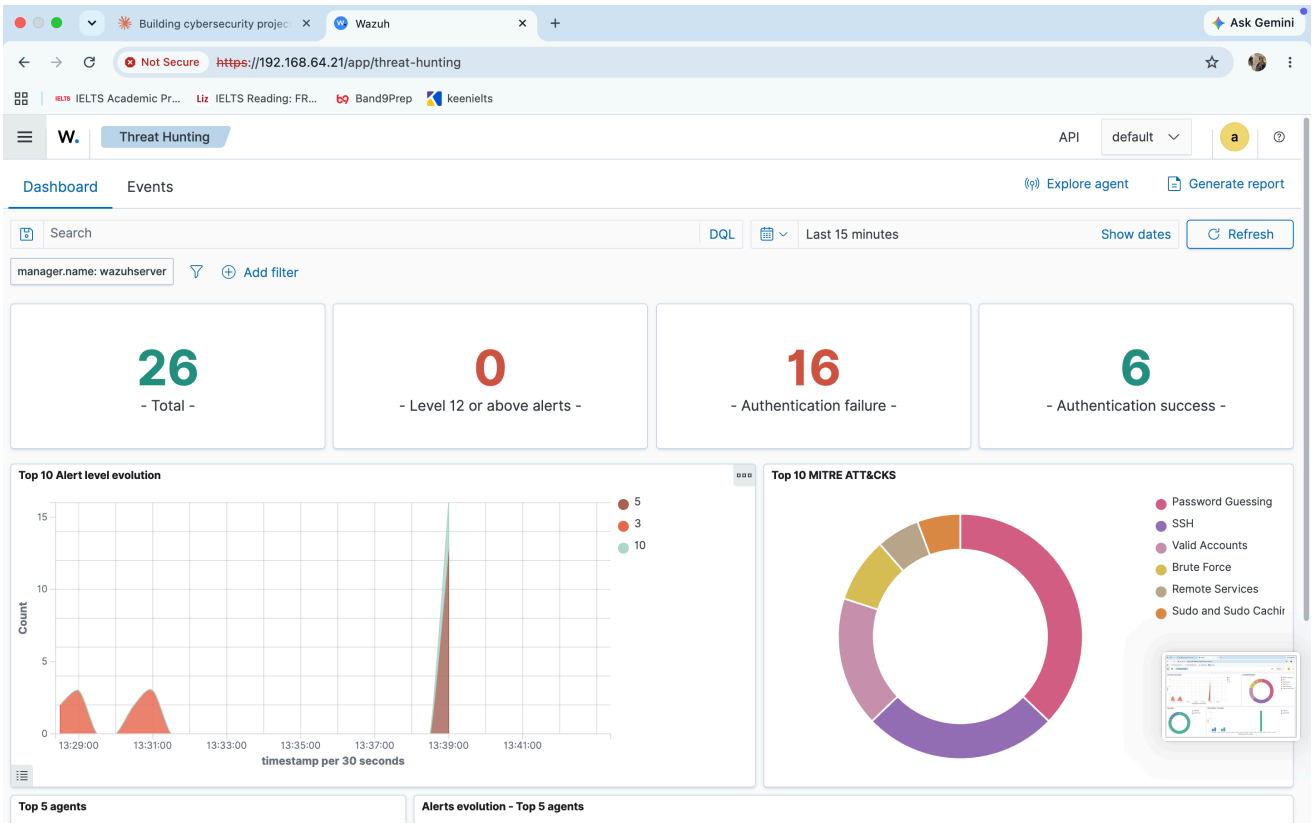


Figure: Wazuh dashboard summary showing the SSH brute-force alerts and MITRE ATT&CK mapping

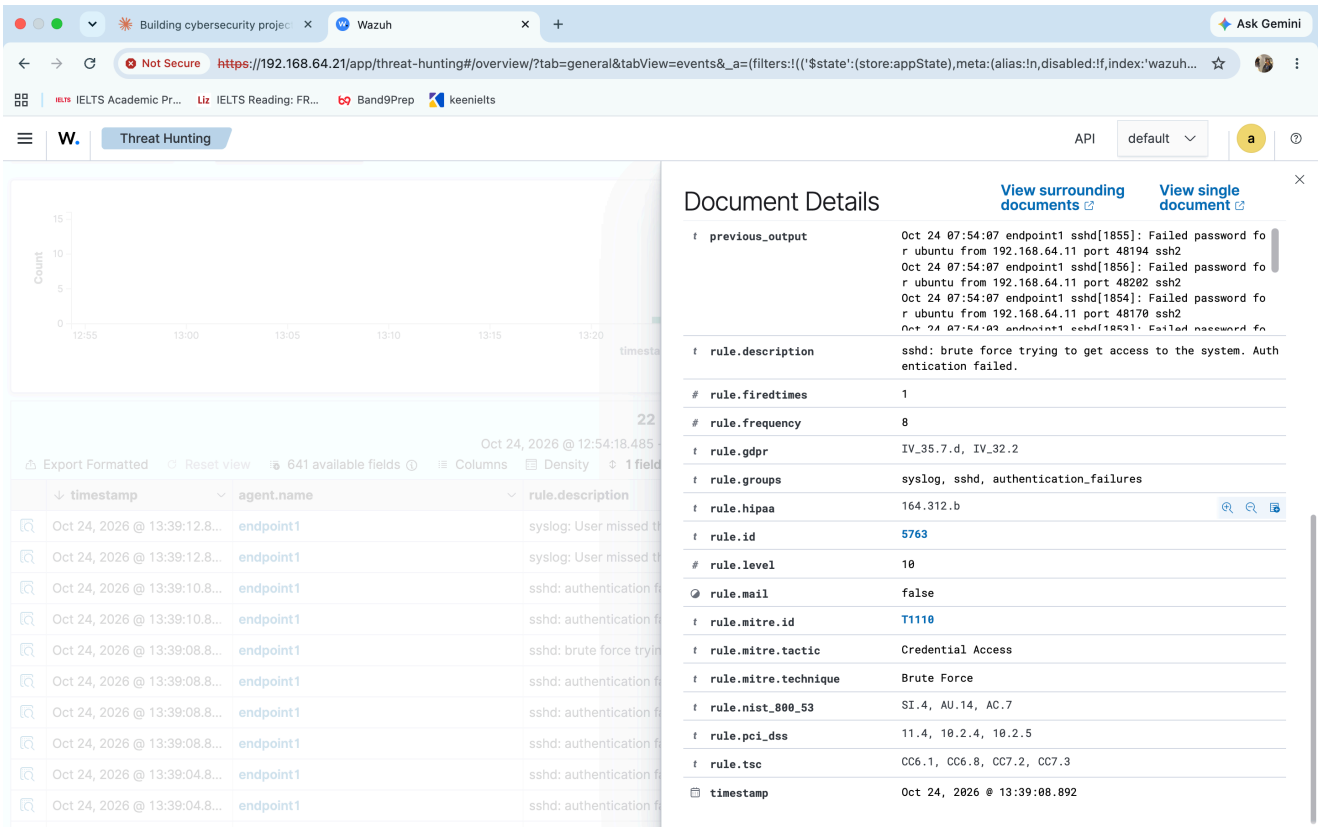


Figure: Expanded alert detail for rule 5763, showing attacker IP, target, and raw log

8.2 Test 2 : Nmap Port Scan (Detection Gap)

An Nmap scan (-sV, and separately -p 1-200) was run against the endpoint. Unlike the SSH attack, a port scan does not generate an authentication failure or any event in the logs that Wazuh's default rule set watches for, so no alert was raised. This was confirmed by searching the dashboard for all events from the attacker's IP during the scan window and finding no matching alerts.

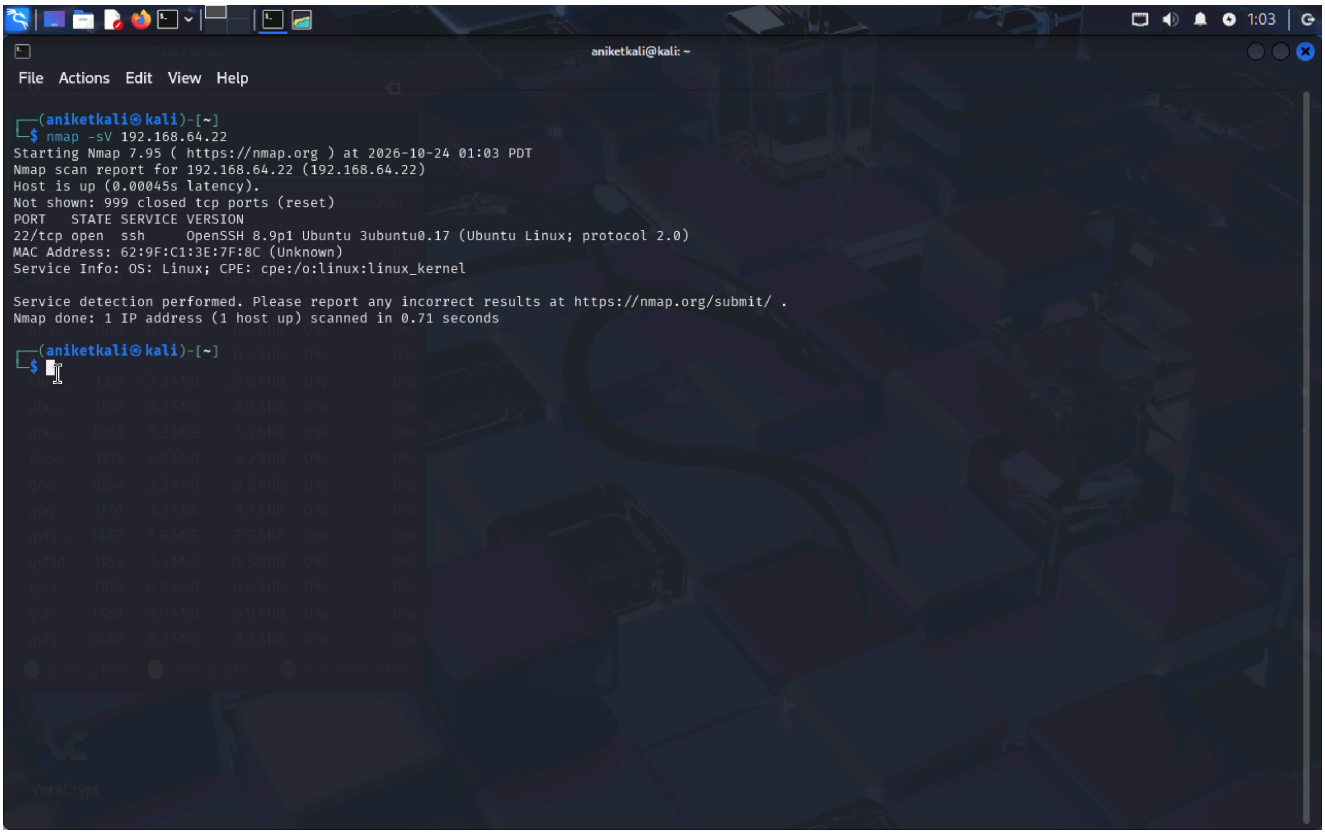


Figure: Nmap scan output against the endpoint

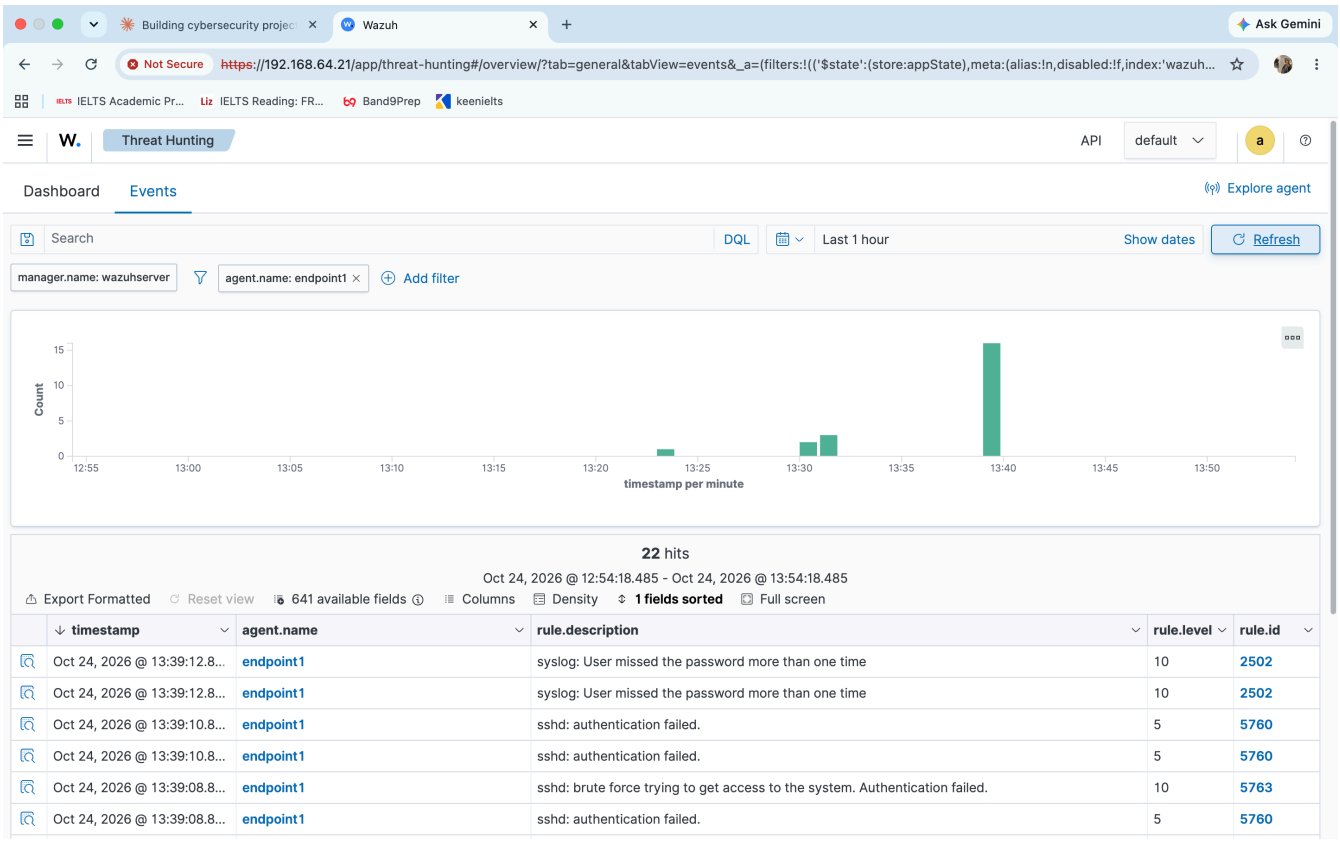


Figure: Wazuh Events view for the scan time window, showing no related alerts

8.3 Test 3 : File Integrity Monitoring (FIM)

File Integrity Monitoring was enabled on a dedicated folder on the endpoint (/home/ubuntu/watched_folder). A file was then created, had its permissions changed to world-writable (777) a change typically associated with suspicious activity modified, and finally deleted, simulating an attacker tampering with and then covering up evidence. All three actions (added, modified, deleted) were detected and logged as separate alerts, correctly attributed to the user who performed them (root, via sudo).

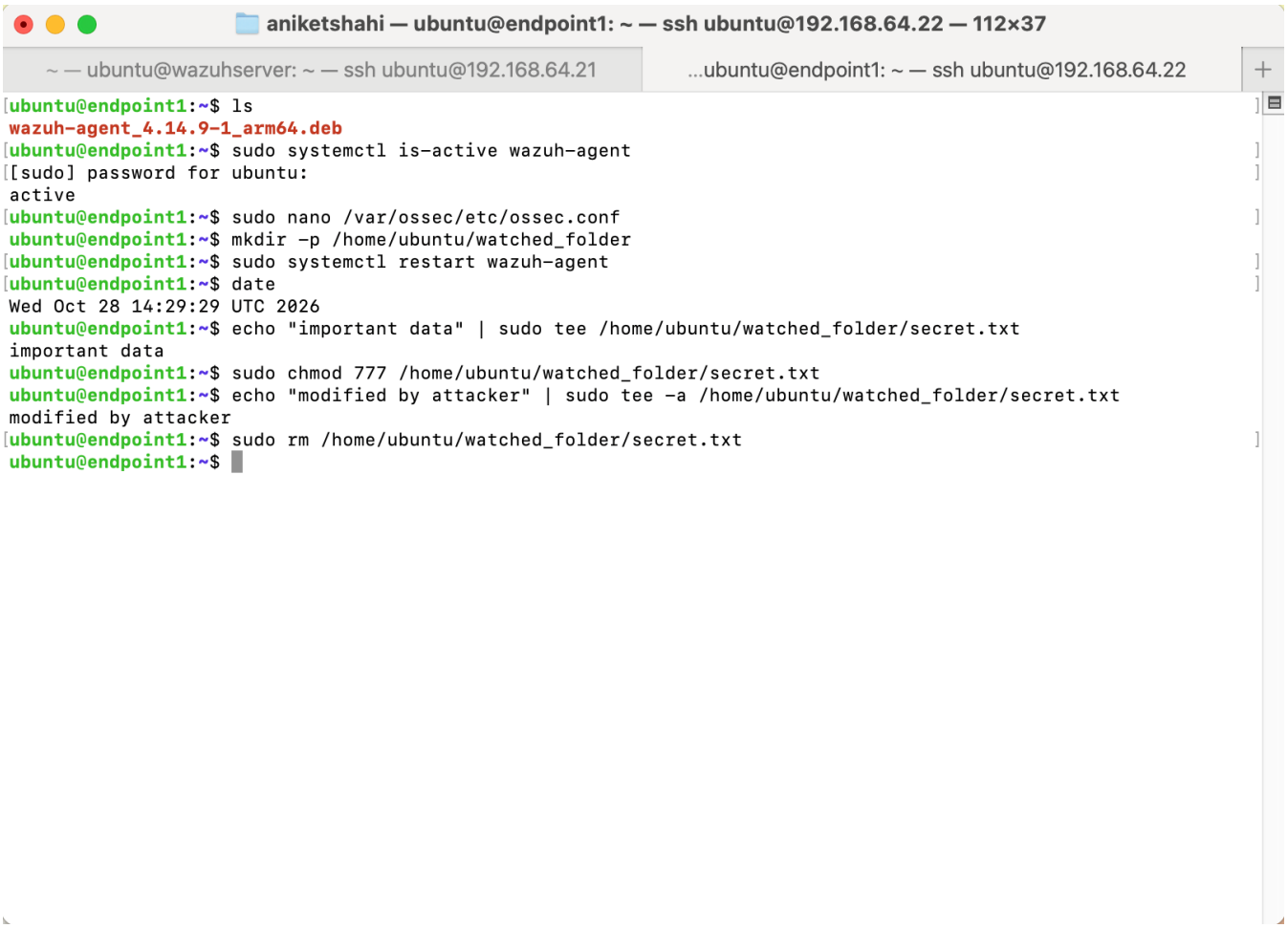


Figure: File tampering commands run on the endpoint

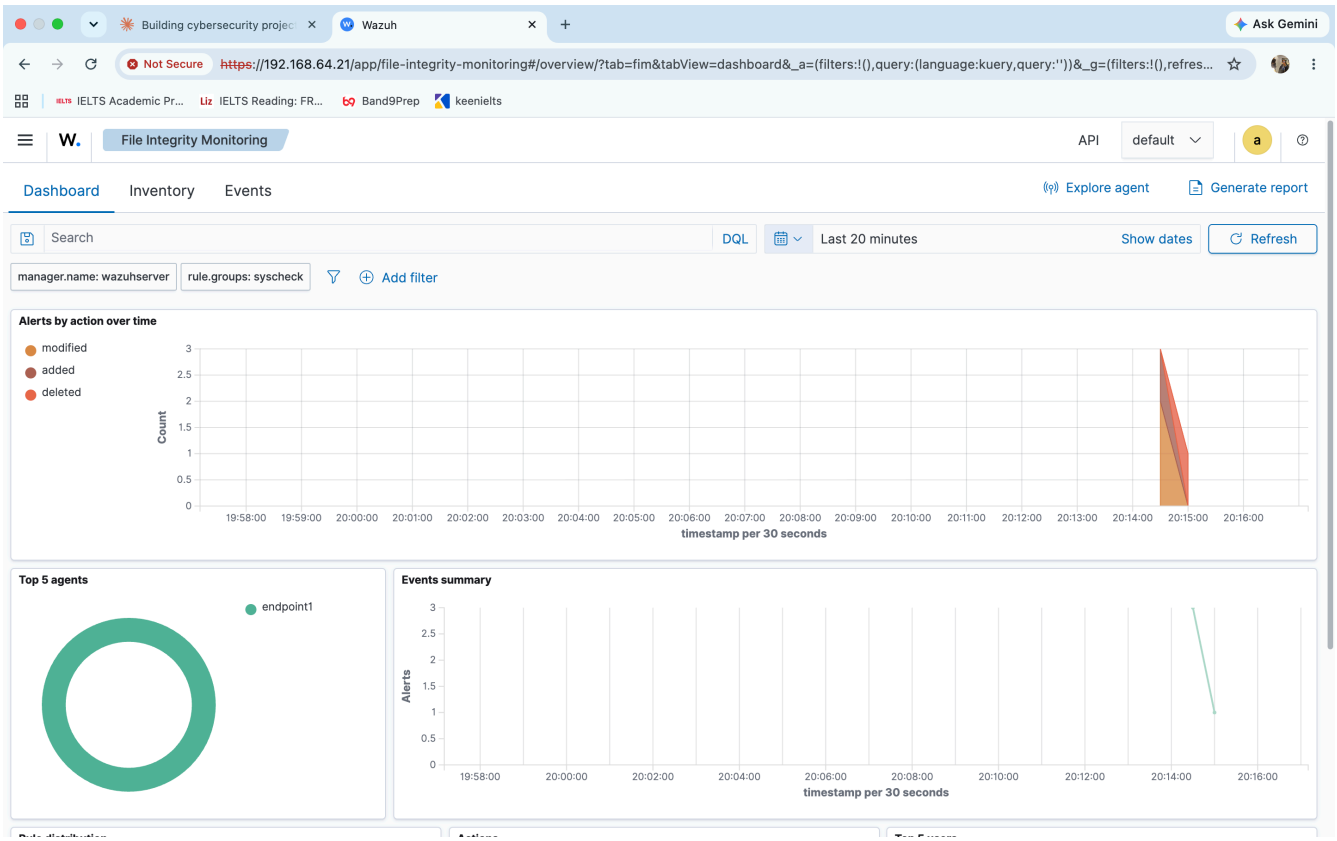


Figure: Wazuh FIM dashboard showing alerts by action (added / modified / deleted)

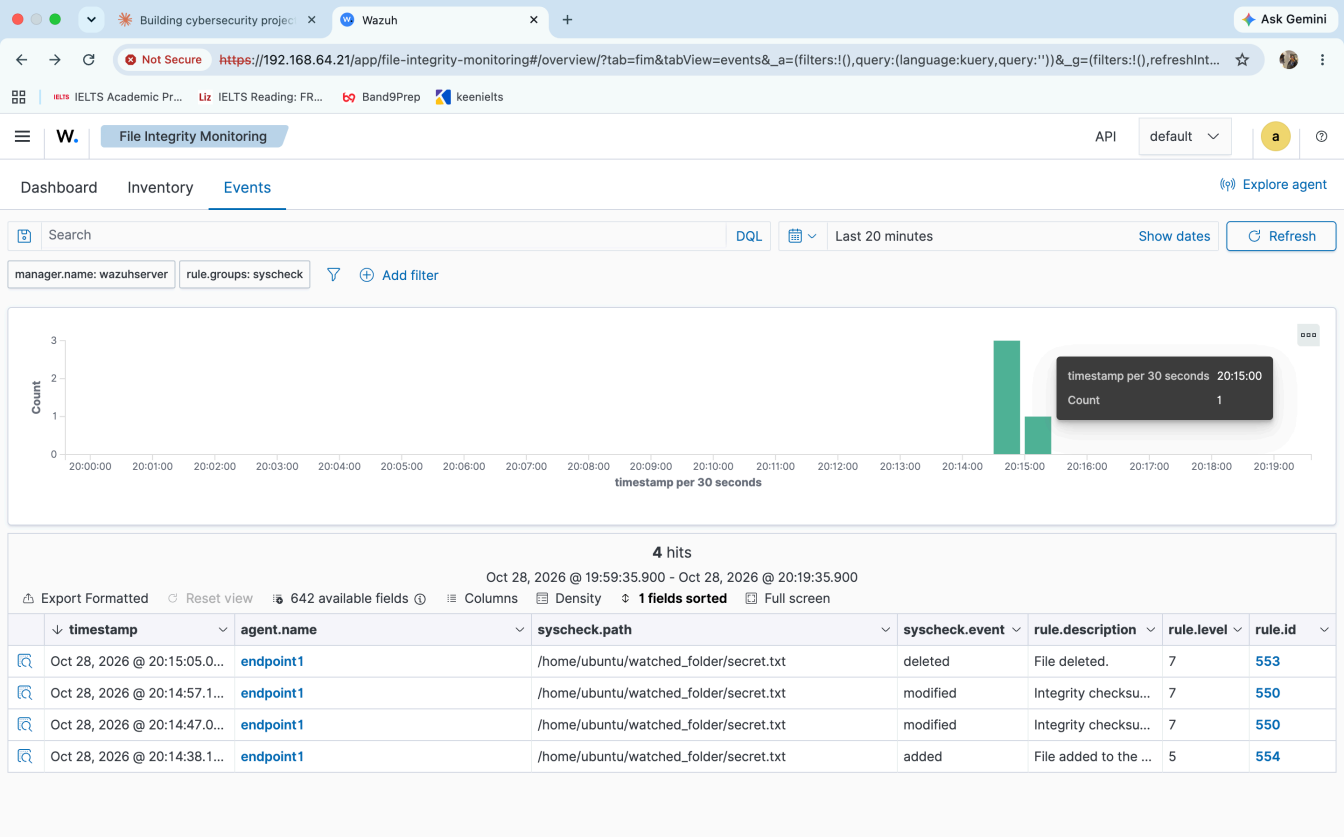


Figure: Expanded FIM alert detail showing the file path and checksum change

8.4 Test 4 : Privilege Escalation (sudo)

A series of sudo commands were run on the endpoint, including attempts to read a sensitive system file. Wazuh logged full accountability for each command (user, working directory, target user, and exact command executed), and specifically flagged the first use of sudo by the account as notable behaviour.

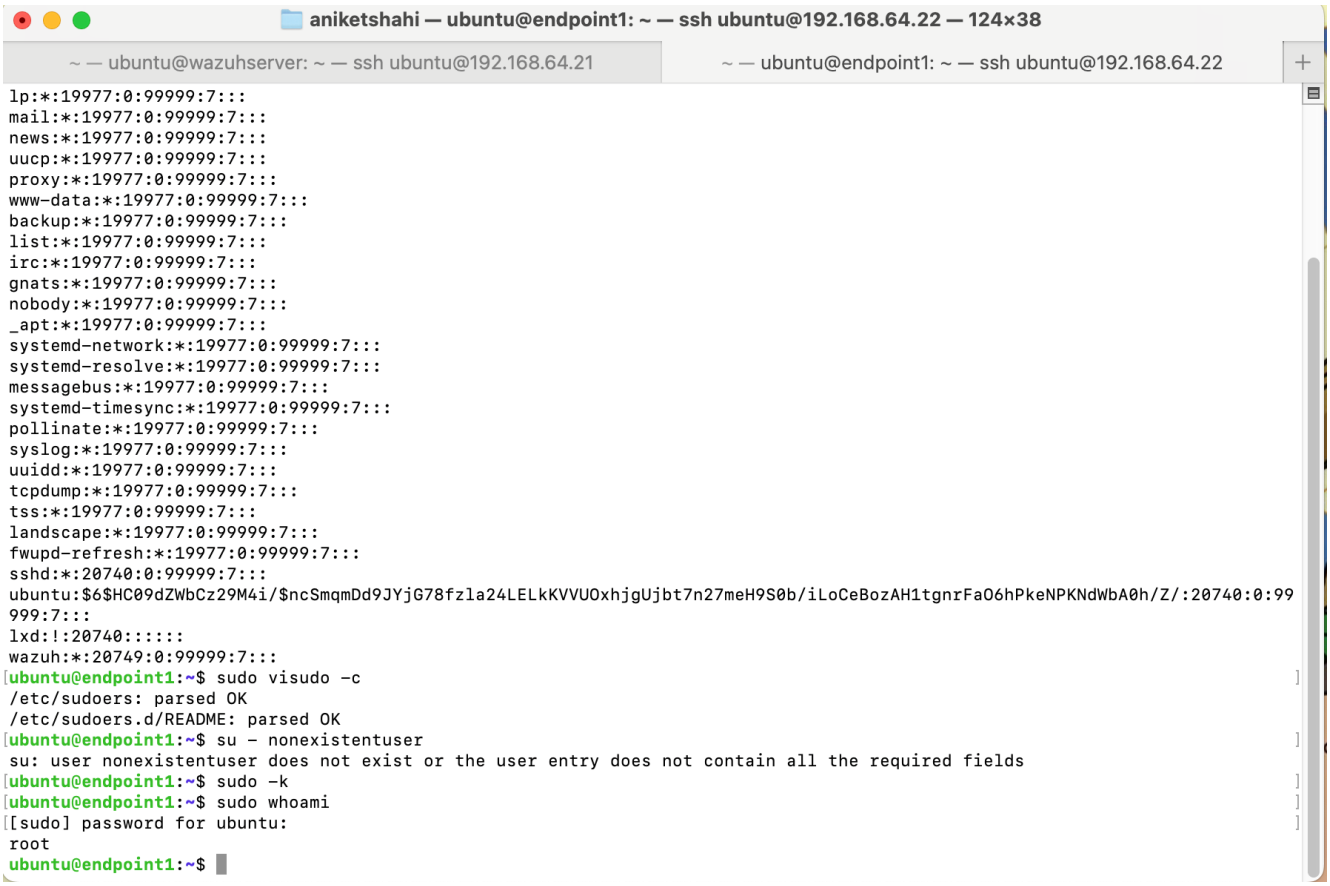


Figure: Privilege escalation / sudo commands run on the endpoint

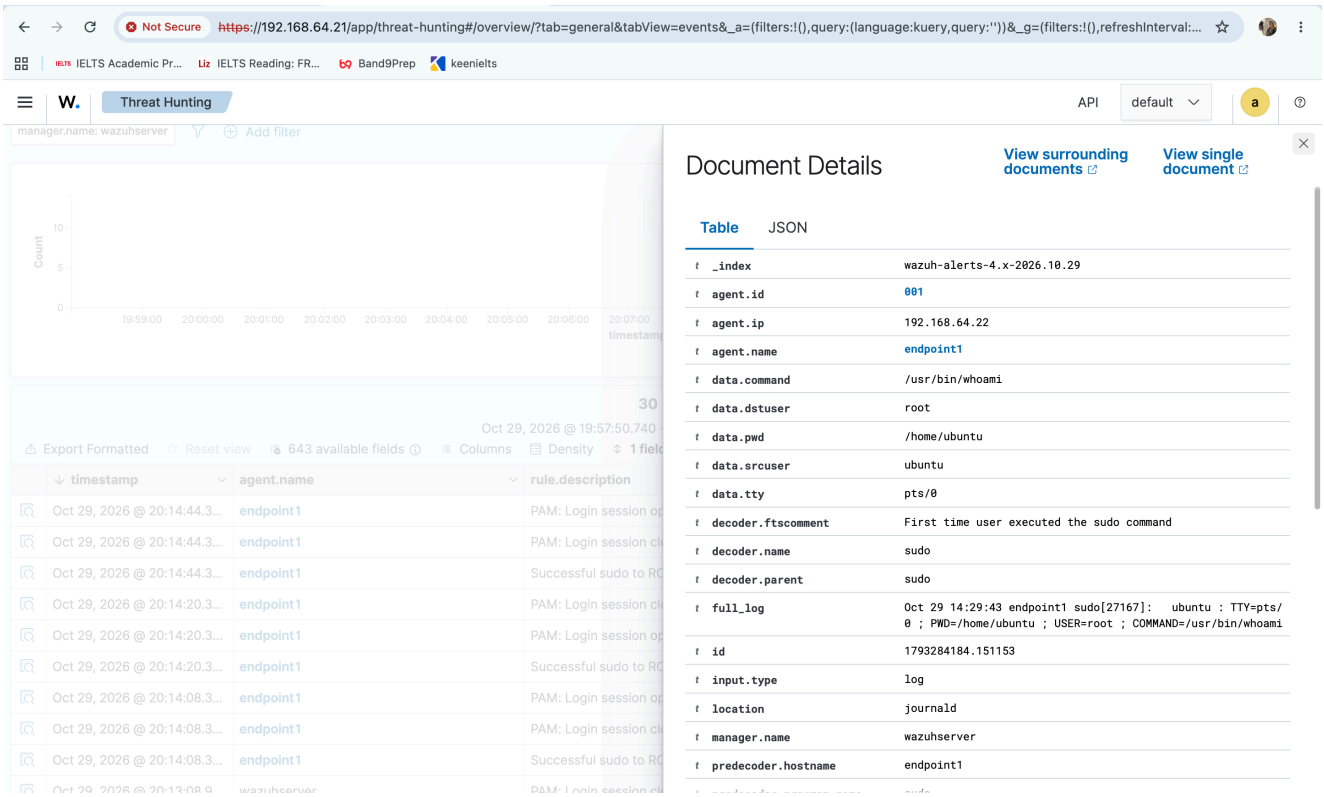


Figure: Wazuh alert detail for the sudo event, showing full command accountability

9. Detection Engineering: Addressing the Nmap Detection Gap

To address the gap identified in Test 2, UFW (Uncomplicated Firewall) was enabled on the endpoint with logging turned on, so that blocked connection attempts would be written to a log file the Wazuh agent could forward. Wazuh's built-in decoder correctly parsed these firewall log lines and matched them against its existing (silent) firewall rule, ID 4100.

A custom rule (ID 100100) was then written to raise an alert whenever eight or more such events arrived from the same source IP within a ten-second window, intended to flag the rapid, multi-port connection pattern characteristic of a port scan, and mapped to MITRE ATT&CK technique T1046 (Network Service Discovery):

The rule's syntax was validated using Wazuh's built-in wazuh-logtest utility, which confirmed the firewall log lines were being decoded correctly and matched against the base rule (4100). After deploying the rule and re-running the Nmap scan, however, the custom frequency rule did not

reliably fire in the dashboard, despite the underlying firewall log entries being confirmed as present and correctly timestamped.

10. Incident Reports:

The following reports are written in the format used by SOC analysts to document detected security events.

Incident Report 1 : SSH Brute Force Attack

- Date/Time: Oct 24, 2026, 13:39:08 UTC
- Reported by: Aniket Shahi
- Affected system: endpoint1 (192.168.64.22)
- Source: 192.168.64.11 (Kali Linux - simulated attacker)
- Summary: Multiple failed SSH login attempts were detected against the local user ubuntu on endpoint1, consistent with an automated password-guessing (brute force) attack.
- Detection: Wazuh rule 5763 (“sshd: brute force trying to get access to the system”) fired after 8 failed attempts within a short window; individual failures were also logged under rule 5760.
- Evidence: Raw log line - Failed password for ubuntu from 192.168.64.11 port 48168 ssh2. Alert level 10.
- MITRE ATT&CK: T1110 - Brute Force (Credential Access tactic)
- Impact: No successful login occurred; the account remained secure throughout the attack.
- Severity: Medium
- Recommended response: Block the source IP at the firewall, enforce SSH key-based authentication instead of passwords, and consider installing fail2ban for automatic blocking after repeated failures.

Incident Report 2 : Unauthorized File Modification (FIM)

- Date/Time: Oct 29, 2026, ~20:14 UTC
- Affected system: endpoint1 (192.168.64.22)
- Summary: A file (secret.txt) was created, had its permissions changed to world-writable (777), modified, and then deleted inside a monitored directory (/home/ubuntu/watched_folder), simulating an attacker tampering with sensitive data to hide evidence.
- Detection: Wazuh File Integrity Monitoring (syscheck) detected all three actions - file added, integrity checksum changed (modified), and file deleted -each as a separate alert.
- Evidence: Alerts show user root, agent endpoint1, and the exact file path with before/after checksums.

- MITRE ATT&CK: Related to T1070 - Indicator Removal (the delete step mimics an attacker covering tracks).
- Impact: None (simulated, controlled test).
- Severity: Medium
- Recommended response: Review why permissions were changed to 777, investigate the user/process responsible, and restrict write access to sensitive directories.

Incident Report 3 : Privilege Escalation via Sudo

- Date/Time: Oct 31, 2026, 20:32:49 UTC (server local time, ~14:29 UTC per agent clock)
- Affected system: endpoint1 (192.168.64.22)
- Summary: The local user ubuntu escalated privileges to root via sudo and executed commands, including a first-time use of sudo by this account, flagged automatically by Wazuh as notable behavior.
- Detection: Wazuh rule 5402 ("Successful sudo to ROOT executed"), plus PAM session open/close events.
- Evidence: Raw log - sudo[27167]: ubuntu : TTY=pts/0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/whoami. Decoder noted: "First time user executed the sudo command."
- MITRE ATT&CK: T1548 - Abuse Elevation Control Mechanism (or can be framed under general privilege escalation).
- Impact: None (authorized user, controlled test).
- Severity: Low to Medium (legitimate account, but worth a baseline alert since it's the account's first sudo use).
- Recommended response: Confirm this sudo usage was expected if unexpected, investigate how the account was accessed and review sudoers policy.

Incident Note 4 : Detection Gap: Nmap Port Scan

- Date/Time: Oct 24 and Oct 31, 2026
- Summary: A standard Nmap scan (nmap -sV and nmap -p 1-200) against endpoint1 produced no alert under Wazuh's default rule set, since the scan generates no authentication failures or file changes - the categories Wazuh watches for out of the box.
- Remediation attempted: Enabled UFW firewall logging on the endpoint and wrote a custom frequency-based rule (ID 100100) to detect 8+ blocked connection attempts from one source within 10 seconds, built on top of Wazuh's built-in firewall rule (4100). Verified via wazuh-logtest that the base firewall events were decoded correctly, but the custom rule did not reliably fire during testing.

11. Results and Analysis:

Of the four attack categories tested, three were detected successfully using Wazuh's default rule set, with no custom configuration required SSH brute force, file integrity violations, and privilege escalation via sudo. This reflects Wazuh strength in monitoring authentication, file system, and command-execution events out of the box.

The one gap identified network-level reconnaissance (port scanning) is a known limitation of host-based log monitoring generally, since a scan leaves little trace in application or authentication logs. The attempt to close this gap using host firewall logging and a custom correlation rule was partially successful (the underlying events were confirmed to be collected and correctly decoded) but did not produce a reliable alert, highlighting a real limitation of this approach versus purpose-built network detection tools.

Overall, the project demonstrates the ability to deploy a working SIEM, interpret its default detections, identify where those detections fall short, and attempt a targeted engineering fix the core cycle of SOC detection engineering work.

12. Limitations and Future Work:

- Only a single Linux endpoint was monitored, a Windows endpoint was not included due to resource constraints (8GB RAM) on the host machine.
- The custom Nmap detection rule did not reliably fire and remains an open item for further investigation.
- The lab ran on a closed virtual network of three VMs, a larger, more realistic environment (multiple endpoints, a domain controller, cloud resources) was out of scope.
- Future work could include integrating the honeypot projects Cowrie logs into the same Wazuh instance for a unified dashboard, adding a Windows endpoint, and replacing the host-firewall-based scan detection with a dedicated network IDS such as Suricata.

13. Conclusion:

This project successfully built and tested a working home SOC lab using Wazuh, covering the full lifecycle from deployment through attack simulation, detection analysis, detection engineering, and incident reporting. It demonstrates practical, hands-on experience directly relevant to SOC analyst and junior security roles: SIEM deployment, log analysis, alert investigation, MITRE ATT&CK mapping, and clear incident documentation. The project also reflects realistic engineering practice, including troubleshooting real installation and detection issues and making a pragmatic decision to document an unresolved gap rather than leave it unaddressed.

14. References:

Wazuh Documentation : https://documentation.wazuh.com/

MITRE ATT&CK Framework : https://attack.mitre.org/

UTM (QEMU for macOS) : https://mac.getutm.app/

Ubuntu Server Documentation : https://ubuntu.com/server/docs